Reliable Underwater Optical Communication Using Smartphones

Mentored by Prof. Arnab Paul

Vishrut Ramraj	2023A7PS0352G
Ragav Krishna Ramesh	2023A7PS0415G

Submitted on March 1, 2026

Contents

2 Abstract **2**

3 Problem Statement **2**

4 High-Level System Architecture **2**

4.1 Architecture Diagram 2

4.2 High-Level Design Explanation 3

5 ROI Detection Algorithm **3**

6 Type 1 and Type 2 Binary Encodings **3**

6.1 Type 1 (T1) Bit Encoding 3

6.2 Type 2 (T2) Bit Encoding 4

6.3 Why ON-Duration Encoding and 100 ms Margins 4

7 Character Encoding (5-bit) **4**

7.1 Exploration: Custom Balanced Huffman Encoding (Discarded Idea) 5

8 Packet Structure **5**

9 Signals / Constant Sequences **6**

10 Protocol FSM **7**

11 Test Mode UI **7**

12 What Actually Works? / Final Deliverables **8**

12.1 Evaluation 8

13 Technical Stack **8**

14 Conclusion **9**

2. Abstract

We built an underwater phone-to-phone optical communication system using only flashlight TX and camera RX. The receiver uses ROI grid analysis, rolling histograms, Otsu confidence, and ON-duration decoding at 20 FPS. The protocol layer includes constants, packet framing, CRC-8, and an FSM for session flow. In the final deliverable, reliable manual packet transfer works for fixed transmitter/receiver positions with 5-character packets.

3. Problem Statement

The goal is reliable underwater communication between two smartphones with no external hardware. The system is constrained by camera frame rate, torch latency, sensor noise, ambient-light variation, and device motion.

Underwater scattering and reflections reduce ON/OFF contrast, and camera auto-exposure can suppress flash differences in a few frames. Therefore, we run a fixed 20 FPS camera path with manual exposure and decode transitions instead of sustained brightness.

The design target is low BER and stable packet decode success under these constraints, using timing margins, CRC checks, and retransmission logic.

4. High-Level System Architecture

This report includes both implemented components and explored approaches from the hackathon. Sections 2–11 cover the full system and attempted protocol path; Section 12 (*What Actually Works? / Final Deliverables*) states what is currently working reliably.

4.1 Architecture Diagram

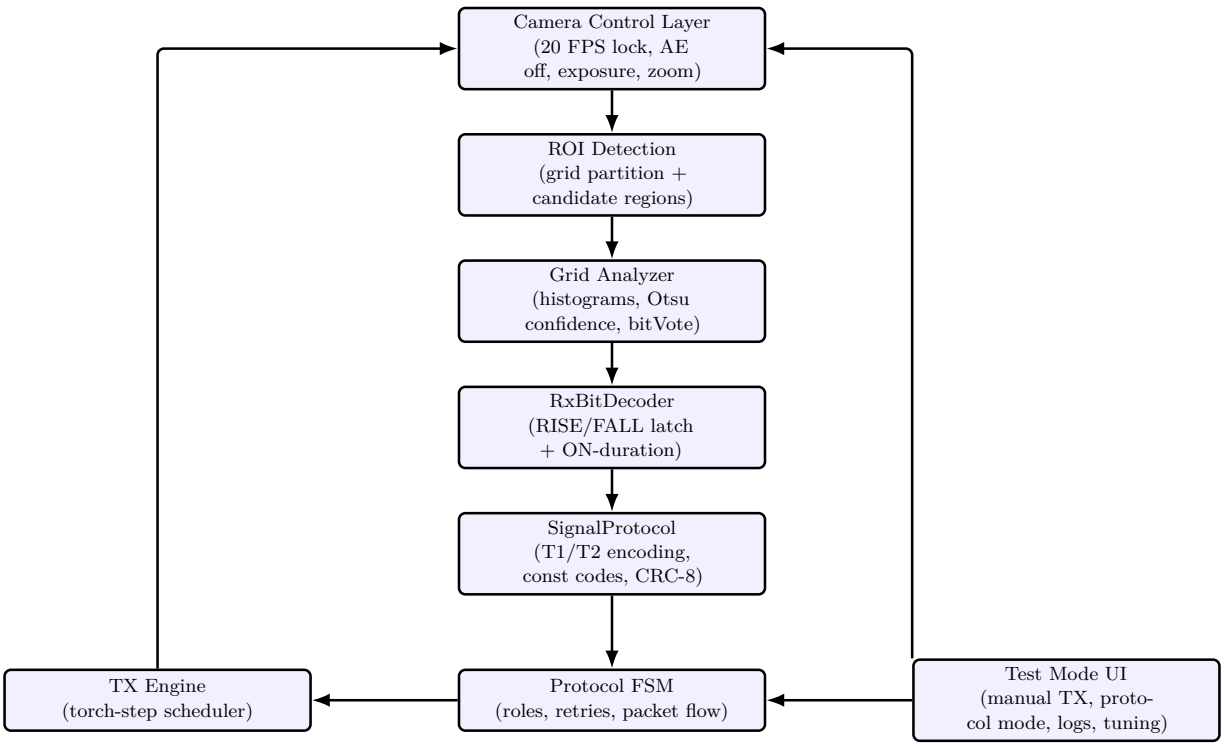

Figure 1: High-level module flow for the underwater communication stack.

4.2 High-Level Design Explanation

The pipeline is layered: camera control stabilizes acquisition, ROI/grid modules estimate signal strength, the bit decoder extracts T1/T2 bits, and protocol modules map bits to constants and packet operations. This separation allows quick tuning of physical parameters (grid, threshold, window, zoom) without changing protocol logic.

The UI supports up to 80 input characters for testing. The current mapping uses lowercase letters plus basic punctuation and space, and the encoding layer can be extended later.

5. ROI Detection Algorithm

At 20 FPS (480p), each frame is split into grids. Each grid keeps a short brightness history; high-confidence connected grids form the active cluster used for decoding.

Per-grid histograms are thresholded with Otsu. Bimodal histograms produce high confidence, while unimodal regions drop toward zero confidence. This suppresses static bright areas and background glare.

The receiver depends on transitions, not sustained ON/OFF states. Sustained states collapse histograms into one mode, reducing confidence and weakening bit votes. Rolling-window mode with $N = 3$ gives sharp transition spikes and stable adapted zero regions, which makes edge timing reliable.

Table 1: Main ROI and decoder hyperparameters used during tuning.

Parameter	Typical value	Practical role
Alpha decay (`alpha`)	0.05	Histogram adaptation speed in alpha-decay mode. Lower values are more stable; higher values react faster.
Confidence threshold	0.15	Minimum Otsu confidence for a grid to be treated as active.
Vote threshold (`VOTE_THRESH`)	0.25	Threshold for bright/dark edge latching in rolling-window mode.
Grid size	8 px	Smaller grids improve spatial precision; larger grids average noise better.
Window size (N)	3 frames	Controls temporal smoothing in rolling-window mode.

Tuning is environment-dependent: low light may need larger grids/lower confidence threshold, while flickering backgrounds may need stricter thresholds. Zoom (1x, 2x, 4x) is a practical SNR control; higher zoom usually helps at longer distances.

6. Type 1 and Type 2 Binary Encodings

6.1 Type 1 (T1) Bit Encoding

Type 1 encodes packet bits with a fixed 600 ms window using ON-duration plus OFF-duration.

Table 2: Type 1 timings.

Bit value	ON duration	OFF duration
0	200 ms	400 ms
1	400 ms	200 ms

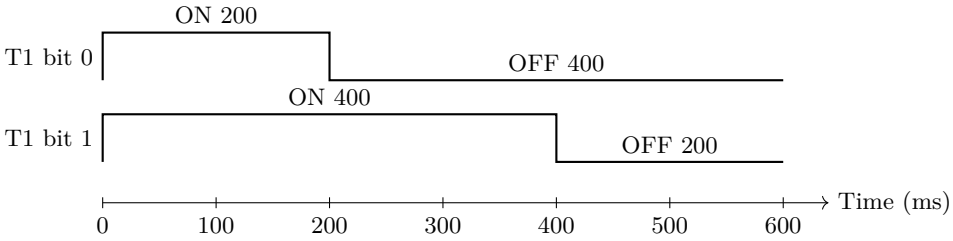

Figure 2: Type 1 ON-duration timing diagram (600 ms per bit).

At 20 FPS (50 ms/frame), 200 ms and 400 ms ON clusters remain separable under frame jitter.

6.2 Type 2 (T2) Bit Encoding

Type 2 is used for constant/control symbols with a 1800 ms bit window.

Table 3: Type 2 timings.

Bit value	ON duration	OFF duration
0	600 ms	1200 ms
1	1200 ms	600 ms

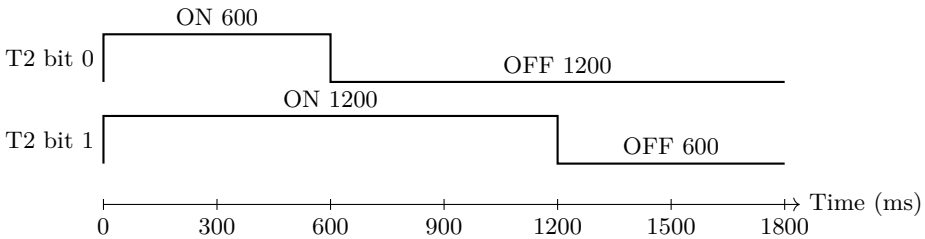

Figure 3: Type 2 ON-duration timing diagram (1800 ms per bit).

6.3 Why ON-Duration Encoding and 100 ms Margins

ON-duration encoding is robust because decoding uses RISE/FALL timing instead of absolute brightness. The 100 ms gaps between timing classes (two frames at 20 FPS) prevent class overlap in normal jitter.

7. Character Encoding (5-bit)

Packet payload uses fixed 5-bit symbols for deterministic timing and simple decoding.

Table 4: 5-bit character mapping used in the current system.

5-bit code / index	Character set	Note
00000 to 11001 (0 to 25)	a to z	Lowercase alphabets
11010 (26)	,	Comma
11011 (27)	.	Period
11100 (28)	!	Exclamation
11101 (29)	?	Question mark
11110 (30)	[space]	Space character
11111 (31)	PAD	Null padding marker

Fixed width keeps packet timing constant and receiver logic simple.

7.1 Exploration: Custom Balanced Huffman Encoding (Discarded Idea)

We explored balanced Huffman coding to reduce average bits per character. It reduced average length in theory, but variable-length symbols introduced variable timing and harder boundary detection. We therefore kept fixed 5-bit symbols.

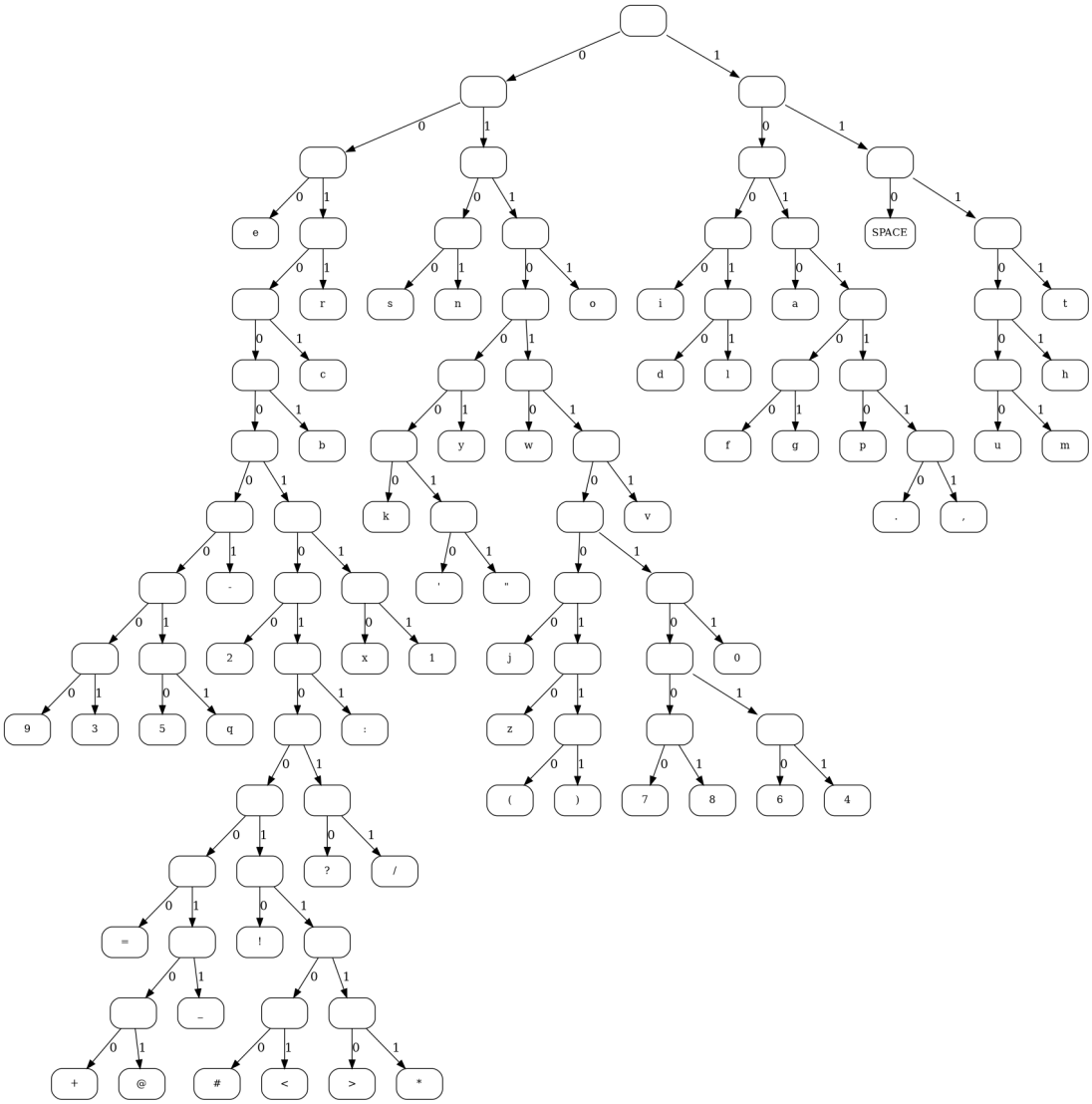

Figure 4: Exploratory balanced Huffman design (not used in final protocol).

8. Packet Structure

Each packet carries 5 characters: 25 data bits + 8 CRC bits = 33 bits total.

C1	C2	C3	C4	C5	CRC-8
25 data bits					8 bits

Figure 5: Packet bit layout: 5 characters + CRC-8.

All packet bits use Type 1 timing, so one packet takes 33×600 ms = 19.8 seconds. PAD fills short inputs to 5 characters.

Example payload: `"hi, ?"`.

```
h = 00111, i = 01000, , = 11010, [space] = 11110, ?  = 11101
```

These data bits are followed by CRC-8 (polynomial $0x07$, CRC8-CCITT) before transmission.

Table 5: Example CRC mismatch behavior for one-bit corruption.

Transmitted packet	Data bits D + CRC C
Received packet	Data bits D' (one bit flipped) + same CRC C
Receiver check	Recomputed CRC $C' \neq C$, packet rejected and retransmitted

9. Signals / Constant Sequences

Control signals use 3-bit Type 2 encoding for robust constant detection.

Table 6: Constant signal definitions.

Signal	3-bit value	Encoding type	Duration
C1	000	T2 (3 bits)	5400 ms
C2	001	T2 (3 bits)	5400 ms
C3	010	T2 (3 bits)	5400 ms
R	011	T2 (3 bits)	5400 ms
S	100	T2 (3 bits)	5400 ms
E	101	T2 (3 bits)	5400 ms
Q	110	T2 (3 bits)	5400 ms

Q:no Special Case

`Q:no` is a mixed-format frame:

- `Q` is transmitted as 3 T2 bits (5400 ms).
- The packet number `no` is transmitted as 4 T1 bits (2400 ms).

Total duration is $5400 + 2400 = 7800$ ms.
Example for requesting packet 3:

```
Q (110 in T2) + 0011 (packet no in T1)
```

T2 is used for a robust constant prefix, while T1 keeps packet-number transmission short.

10. Protocol FSM

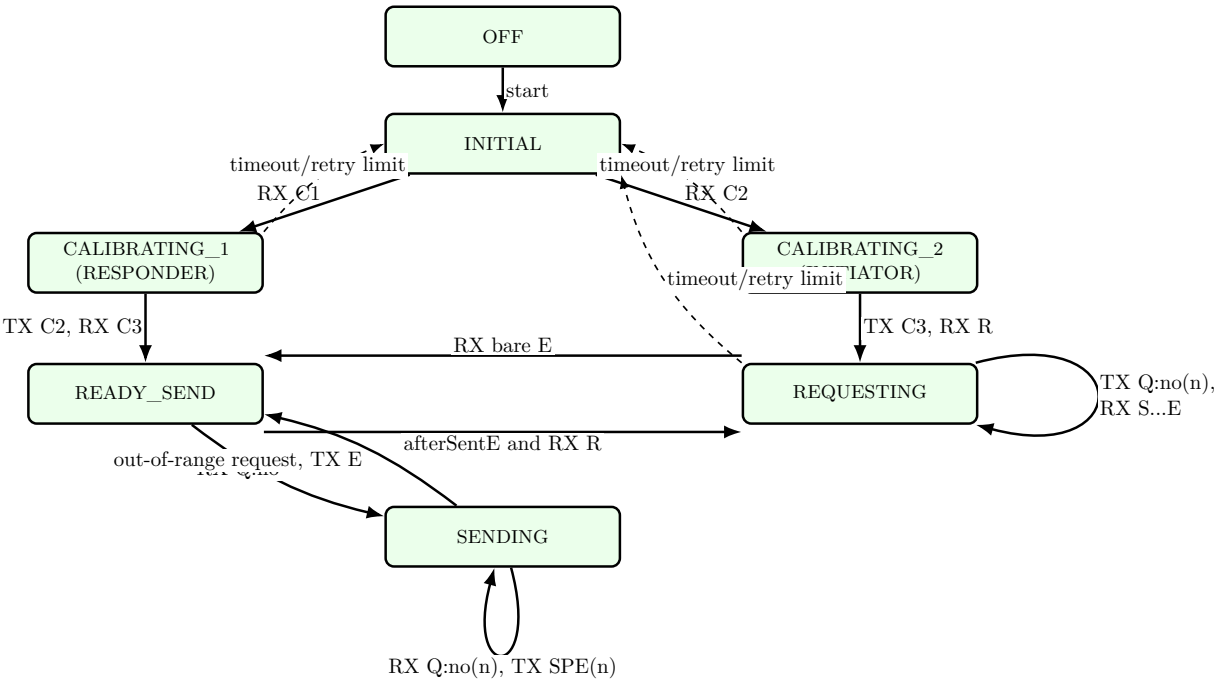

Figure 6: Protocol finite state machine for session setup and packet exchange.

The FSM separates roles: responder path (`C1` detected) reaches `READY_SEND`, and initiator path (`C2` detected) reaches `REQUESTING`. This prevents simultaneous data transmission.

In `REQUESTING`, nodes send `Q:no`; in `SENDING`, nodes answer with `SPE` or bare `E`. `RESPONSE_DELAY_MS` (3000 ms) is essential to avoid TX/RX overlap.

11. Test Mode UI

Test Mode UI Placeholder
Place `test_mode_ui.png` in project root.

Figure 7: Test mode interface used during development and evaluation.

The test UI supports manual signals, packet transmission, and full protocol mode in one screen.

Table 7: UI blocks and their debugging purpose.

UI feature	Use in debugging and evaluation
Manual signal transmit	Sends constants (`C1/C2/C3/R/S/E/Q`) to validate timing classes and edge detection.
Packet transmit	Sends packet-form payloads to test 5-bit mapping, CRC behavior, and packet duration.
Full protocol controls	Runs the FSM flow end-to-end and validates retries, role changes, and completion handling.
Sliding window / mode inputs	Switches histogram mode and window length to compare transition stability.
Event log and signal log	Gives a timestamped trace for decoded symbols, state changes, and timeout events.
Alpha / threshold inputs	Tunes confidence and edge trigger behavior under different lighting conditions.
Grid and zoom controls	Adjusts spatial sampling and SNR in real time (1x, 2x, 4x).

It is the main debugging surface for reproducible timing and state inspection.

12. What Actually Works? / Final Deliverables

The report includes all protocol work attempted during the hackathon, but the full automated protocol loop was not fully stabilized within the timeline.

What works reliably now is packet TX/RX with end-to-end decoding between fixed transmitter and receiver devices. We can transmit up to 80 characters by splitting the message into packets of 5 characters each and decoding all packets at the receiver.

This behavior is consistent with the current `SignalProtocol` implementation:

- `MAX_MESSAGE_LEN = 80`, `CHARS_PER_PACKET = 5`, and `messageToPackets()` split long input into 5-character chunks (up to 16 packets).
- `encodePacket()` constructs each 33-bit packet (25 data bits + 8 CRC bits) using Type 1 timing.
- `checkPacketCrc()` validates integrity at the receiver.
- `decodePacket()` converts validated bits back into the 5-character decoded output.

Final deliverable: a working underwater optical link that reliably sends messages up to 80 characters via packetized 5-character transfer between fixed devices.

12.1 Evaluation

We evaluated approximately 40 manually transmitted 5-character packets. All were decoded correctly, giving observed BER = 0 for this test set.

Demo link: CLICK HERE

13. Technical Stack

The implementation uses Kotlin + CameraX 1.4.0 + Camera2Interop, with direct activity-based control (no MVVM, Dagger, or Hilt) for timing-critical iteration.

Table 8: Core technical stack.

Layer	Stack choice
Language	Kotlin
Camera API	CameraX 1.4.0 + Camera2Interop (AE off, explicit frame duration, torch control)
Concurrency	HandlerThreads for TX scheduling and camera frame processing
Signal processing	Rolling-window histograms, Otsu thresholding, confidence-weighted bit voting
Encoding	Custom T1/T2 ON-duration binary scheme + 5-bit character map
Integrity	CRC-8 (CRC8-CCITT, polynomial 0x07)
Protocol	Custom half-duplex constant-code + packet-request FSM stack

The stack is intentionally minimal to preserve timing control and observability.

14. Conclusion

This project demonstrates a practical smartphone-only underwater optical link. The core gains come from transition-based decoding, ON-duration timing classes, fixed 20 FPS capture, and CRC-based validation.

The automated full protocol path is still under refinement, while manual packet transfer is stable in current tests. Next steps are in-water retuning, broader BER characterization, and protocol hardening for fully automated exchange.